Artificial Intelligence & Neural Networks

High-Yield Exam Crash Course • Comprehensive Quick-Study Notes

> ⚡ **3-Hour Exam Strategy:** This document directly translates your official university syllabus and your specific high-priority exam questions into clear, bulleted answers. Focus on the structural diagrams and comparison tables—they carry the highest marks!

Module 1: Foundations of Artificial Intelligence & Agents

1. What is Artificial Intelligence (AI)?

Artificial Intelligence is a branch of Computer Science concerned with building smart, automated systems capable of performing tasks that typically require human intelligence. These tasks include visual perception, speech recognition, decision-making, and natural language translation.

Key Features of AI

- **Deep Learning & Reasoning:** Emulating human cognitive properties to evaluate complex data and reach reliable decisions.
- **Data Ingestion & Adaptability:** Learning continuously from dynamic external environment variations using incoming high-volume datasets.
- **Problem-Solving Capacities:** Utilizing search patterns, logical processing, and specialized heuristics to solve multi-variable puzzles.
- **Automation & Pattern Extraction:** Offloading repetitive analytical processes to background workers without manual intervention.

Core Goals of AI

1. **To Create Expert Systems:** Developing robust software engines that exhibit intelligent behavior, explain complex scenarios, and advise users.
2. **To Implement Human Intelligence in Machines:** Creating cognitive architectures that think, learn, and act like humans (Rational Action).
3. **Perception and Ingestion:** Creating systems that fluidly process natural sensory feedback (Computer Vision, Natural Language Processing).

Real-World Applications

- **Natural Language Processing (NLP):** Automated customer response bots, smart spelling engines, text translation systems.

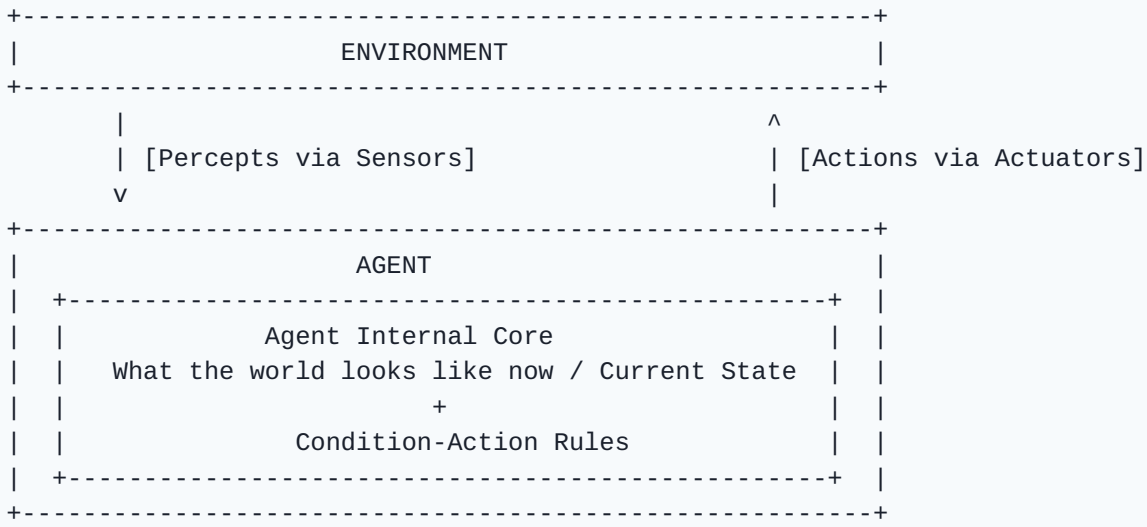
- **Computer Vision:** Automated medical screening (X-ray classification), factory fault-detection lines, biometric passport checkpoints.
- **Autonomous Systems:** Real-time delivery tracking optimization engines, precision aerial drones, and visual lane-keeping vehicle driver assists.
- **Healthcare Diagnostics:** Predictive metrics mapping cancer growth anomalies via genetic patterns and historic CT-scan libraries.

> 💡 **High-Score Exam Definition:** Always mention **Stuart Russell and Peter Norvig's** 4 pillars of AI: Systems that (1) Think like humans, (2) Act like humans, (3) Think rationally, and (4) Act rationally.

2. Intelligent Agents & Structural Types

An **Agent** is anything that perceives its **Environment** through **Sensors** and acts upon that environment through **Actuators**. It relies on a continuous loop: *Percepts* → *Processing* → *Action*.

The Base Agent Architecture



The 5 Core Agent Architectures (Syllabus Core)

A. Simple Reflex Agent

Acts based *only* on the current percept, completely ignoring historical trends. It runs on static **Condition-Action Rules** (If-Then statements).

- **Example:** A simple automated thermostat. If Temperature > 25°C → Turn on cooling.

```
[Sensor Input] ---> [What World Is Now] ---> [Condition-Action Rules] ---> [Actuator Output]
```

B. Model-Based Reflex Agent

Maintains an internal state to keep track of aspects of the environment that cannot be viewed right now (handles partial visibility).

- **Example:** An autonomous car tracking a hidden vehicle using a background distance-history vector.

```
[Sensors] -> [Internal State (History)] -> [How World Evolves] -> [Rules] -> [Actuators]
```

C. Goal-Based Agent

Combines current status data with explicit goal targets to select the path that leads directly to the desired state.

- **Example:** Global Positioning System (GPS) route-finding engines choosing a clear path to a specified destination coordinates.

D. Utility-Based Agent

Measures actions using a continuous scale (utility function) to check how "happy" or efficient the final outcome state is.

- **Example:** A financial investment bot evaluating portfolio options to balance trade-off metrics between high returns and risk exposure.

E. Learning Agent

Separated into four distinct components to adapt to unknown environments: Learning Element (makes improvements), Critic (evaluates performance against a fixed metric), Learning Goal Generator (suggests novel actions), and Performance Element (selects external actions).

- **Example:** An adaptive speech recognition assistant learning your accent over multiple sessions.

Module 2: Informed & Uniformed Search Strategies

1. Breadth-First Search (BFS) vs. Depth-First Search (DFS)

These are the two fundamental un-informed search algorithms used to traverse graph and tree structures in AI systems.

Comparison Criterion	Breadth-First Search (BFS)	Depth-First Search (DFS)
Core Mechanism	Explores the graph layer by layer, visiting all neighboring sibling nodes before moving deeper.	Explores as deep as possible along each branch before backtracking up to the next sibling.
Data Structure	Uses a **First-In-First-Out (FIFO) Queue**.	Uses a **Last-In-First-Out (LIFO) Stack** (or recursion).
Time Complexity	$O(b^d)$ where b is the branching factor, d is depth.	$O(b^m)$ where m is max depth of any path.
Space Complexity	$O(b^d)$ — Highly resource heavy; keeps every discovered node in memory.	$O(b \times m)$ — Low memory footprint; stores only the current path nodes.
Optimality	**Yes**; guaranteed to locate the shallowest path to a goal node.	**No**; can find a deeper goal node first, skipping better options.
Infinite Loops?	Safe from infinite loops in finite graphs.	Can get trapped in infinite paths if loops aren't tracked.

> ⚠️ **Exam Note on Complexities:** Make sure to label your variables clearly: b = Branching Factor, d = Depth of shallowest solution, m = Maximum path depth.

2. Informed Search: The A* Search Algorithm

The A* algorithm combines the actual distance traversed from the start node with an estimated heuristic cost to the destination. It is an optimal and complete informed search algorithm.

The total evaluation function is calculated using:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = The exact actual path cost incurred to reach node n from the starting state.
- $h(n)$ = The estimated heuristic cost to travel from node n to the closest goal node.

Conditions for Optimality

1. **Admissibility:** The heuristic $h(n)$ must *never overestimate* the actual cost to reach the goal. It must be optimistic or exactly accurate ($h(n) \leq h^*(n)$).

2. **Consistency (Monotonicity):** For every node n and every successor n' generated by an action a, the estimated cost must obey the triangle inequality: $h(n) \leq c(n, a, n') + h(n')$.

Module 3: Adversarial Search & Games

1. Alpha-Beta Pruning

Alpha-Beta pruning is an optimization technique applied to the classic **Minimax game tree algorithm**. It removes branches that cannot possibly influence the final decision of the players, reducing computation time.

The Strategic Variables

- **Alpha (α):** The highest (best) choice value discovered so far along the path for the **MAX** player. Initialized to -∞.
- **Beta (β):** The lowest (best) choice value discovered so far along the path for the **MIN** player. Initialized to +∞.

The Pruning Condition: During evaluation, if at any point $\alpha \geq \beta$, the remaining sub-tree under that specific node is completely skipped (pruned).

```
            [ MAX Node ]  --> Maintains Alpha (α)
            /                  [ MIN Node ]     [ MIN Node (Pruned here if α >= β) ]
       /             (Value: 4)   (Value: 5)
```

Key Properties

- **Does it change the final outcome?** No, Alpha-Beta pruning yields the exact same move as standard Minimax; it just does it much faster.
- **Best-Case Time Complexity:** Pruned down to $O(b^{m/2})$ with perfect move ordering, allowing the search to explore twice as deep as basic Minimax in the same amount of time.

Module 4: Machine Learning & Deep Learning (The Neural Network Shift)

1. Machine Learning vs. Deep Learning Architecture

Understanding the exact conceptual boundary between classical Machine Learning and modern Neural Deep Learning frameworks is essential for answering core design questions.

Feature Matrix	Machine Learning (ML)	Deep Learning (DL)
Data Dependency	Works well on medium or small tabular datasets. Performance plateaus with more data.	Requires massive training datasets to optimize millions of internal weight parameters without overfitting.
Feature Extraction	**Manual:** Requires domain experts to clean, isolate, and explicitly select input variables.	**End-to-End:** Layers automatically extract deep representations directly from raw inputs (pixels, text).
Hardware Prerequisites	Can be executed comfortably on standard, consumer-grade CPU architectures.	Requires massive multi-core hardware accelerators like modern GPUs/TPUs for tensor math.
Execution Timelines	Quick training phases (minutes to hours), but testing execution scales instantly.	Extremely long training phases (days or weeks), but runs inference very quickly once optimized.
Core Algorithms	Linear Regression, Decision Trees, Naive Bayes Classifier, Support Vector Machines (SVM).	Multi-Layer Perceptrons (MLP), Convolutional Neural Networks (CNN), Recurrent Nets (RNN), Transformers.

```
Classical ML:  [Input Data] ---> [Manual Feature Engineering] ---> [Classifier ML Engine] --->

Deep Learning: [Input Data] ------------------------> [Deep Layered Neural Net] -------------
```

2. Essential Mathematical Classifiers

Naive Bayes Classifier

A supervised learning algorithm based on **Bayes' Theorem** with the "naive" assumption of conditional independence between every pair of features given the class variable.

$$P(C \mid X) = [\ P(X \mid C) \times P(C)\ ] / P(X)$$

Expectation-Maximization (EM) Algorithm

An iterative optimization method used to estimate maximum likelihood parameters in statistical models where variables are latent or data is missing.

- **Expectation (E-Step):** Computes the expected log-likelihood of the hidden parameters using current data estimates.

- **Maximization (M-Step):** Calculates new parameter values that maximize the expected log-likelihood found during the E-Step.

Reinforcement Learning (RL)

An agent learns how to behave in an environment by performing actions and seeing the results. It works through a system of trial and error, guided by rewards or penalties.